# 학습 노트: LangGraph StateGraph 5-노드 파이프라인 (Dev2)

작성일: 2026-05-24
관련 파일: [dev2/schema.py], [dev2/nodes/run_tools.py], [dev2/nodes/vector_search.py], [dev2/nodes/llm_judge.py], [dev2/nodes/store.py]
복습 예정: D+1, D+3, D+7

## 한 줄 요약

LangGraph StateGraph는 "공장 컨베이어 벨트"다. 각 작업대(노드)가 부품(state)을 받아 자기 일을 한 뒤 다음 작업대로 넘기고, 우리는 작업대 순서(엣지)만 그려주면 알아서 흐른다.

---

## 1. StateGraph 기본 구조 — add_node / add_edge / compile()

비유: 레고 조립 설명서를 만드는 것. 블록(노드)을 등록(add_node)하고,
"이 블록 다음에 저 블록"을 화살표(add_edge)로 잇고, 마지막에 "조립 완료(compile)"를 누르면
실행 가능한 그래프 객체가 나온다.

핵심: [StateGraph(StateType)]로 그래프를 만들고 -> [add_node("이름", 함수)]로 작업대 등록 ->
[add_edge("A", "B")]로 순서 연결 -> [compile()]로 실행 가능한 객체를 얻는다.
compile() 전에는 "설계도", 후에는 "실행기"다.

```
[python]
from langgraph.graph import StateGraph
from dev2.schema import Dev2State
from dev2.nodes.run_tools import run_tools_node
from dev2.nodes.vector_search import vector_search_node

builder = StateGraph(Dev2State)          # 1) State 타입 지정
builder.add_node("run_tools", run_tools_node)        # 2) 작업대 등록
builder.add_node("vector_search", vector_search_node)
builder.add_edge("run_tools", "vector_search")       # 3) 순서 연결
graph = builder.compile()                # 4) 실행기 완성
```

---

## 2. TypedDict State — dict 반환 시 merge 동작

비유: 화이트보드(state) 하나를 여러 사람이 돌려가며 쓴다.
각 노드는 자기가 채운 칸만 적은 메모(dict)를 제출하고, LangGraph가 화이트보드의 해당 칸만 덮어쓴다.
적지 않은 칸은 그대로 보존된다.

핵심: 노드는 전체 state가 아니라 바뀐 필드만 담은 dict를 반환한다.
LangGraph는 기본 reducer로 이 dict를 기존 state에 얕은 병합(shallow merge)한다
(같은 키는 덮어쓰기, 없던 키는 추가). 그래서 [run_tools_node]가
[{"static_raw": [...], "security_raw": [...]}]만 반환해도 [pr_diff] 같은 기존 필드는 유지된다.

```python
# dev2/nodes/run_tools.py 의 실제 패턴
async def run_tools_node(state: Dev2State) -> dict:
    diff = state.get("pr_diff", "")          # 기존 state 읽기
    ...
    # 바뀐 필드만 반환 -> LangGraph가 state에 merge
    return {"static_raw": static_raw, "security_raw": security_raw}
```

주의: 기본 동작은 덮어쓰기다. 리스트를 누적하려면 [Annotated[list, operator.add]]로 reducer를 따로 지정해야 한다 (Dev2는 단계별 덮어쓰기라 기본 동작으로 충분).

---

## 3. START / END 상수 — 시작/종료 연결

비유: 컨베이어 벨트의 "입구(START)"와 "출구(END)" 표지판.
입구에서 첫 작업대로, 마지막 작업대에서 출구로 화살표를 그어야 흐름이 완성된다.

핵심: [langgraph.graph]에서 [START], [END]를 import한다.
[add_edge(START, "첫노드")]로 진입점을, [add_edge("마지막노드", END)]로 종료점을 지정한다.
([set_entry_point] / [set_finish_point]도 같은 역할의 구버전 API다.)
START/END를 빼먹으면 "진입점이 없다"는 컴파일 에러가 난다.

```python
from langgraph.graph import StateGraph, START, END

builder = StateGraph(Dev2State)
builder.add_node("run_tools", run_tools_node)
builder.add_node("store", store_node)
builder.add_edge(START, "run_tools")     # 입구 -> 첫 노드
builder.add_edge("store", END)           # 마지막 노드 -> 출구
```

---

## 4. 5-노드 선형 파이프라인 패턴

비유: 5단계 공장 라인. 원자재 검사(run_tools) -> 과거 불량 사례 조회(vector_search)
-> 검사관 판정(llm_judge) -> 심각 건 보고(escalate) -> 창고 입고(store).
한 줄로 쭉 이어지는 직선 라인이라 분기가 없다.

핵심: 노드들을 순서대로 [add_edge]로 줄줄이 잇기만 하면 선형 파이프라인이 된다.
각 노드는 앞 노드가 채운 state 필드를 입력으로 받고, 다음 노드가 쓸 필드를 채워서 dict로 반환한다.
Dev2는 [static_raw/security_raw] -> [enriched] -> (judged) -> final_output 순으로 state가 풍부해진다.

```python
from langgraph.graph import StateGraph, START, END
from dev2.schema import Dev2State
from dev2.nodes.run_tools import run_tools_node
from dev2.nodes.vector_search import vector_search_node
from dev2.nodes.llm_judge import llm_judge_node
from dev2.nodes.escalate import escalate_node      # Critical Slack 알림
from dev2.nodes.store import store_node

def build_dev2_graph():
```

```
    b = StateGraph(Dev2State)
    b.add_node("run_tools", run_tools_node)
    b.add_node("vector_search", vector_search_node)
    b.add_node("llm_judge", llm_judge_node)
    b.add_node("escalate", escalate_node)
    b.add_node("store", store_node)

    b.add_edge(START, "run_tools")
    b.add_edge("run_tools", "vector_search")
    b.add_edge("vector_search", "llm_judge")
    b.add_edge("llm_judge", "escalate")
    b.add_edge("escalate", "store")
    b.add_edge("store", END)
    return b.compile()
```

---

## 5. graph.ainvoke() vs graph.invoke()

비유: invoke는 "전화 받을 때까지 수화기 들고 기다리기"(동기),
ainvoke는 "메시지 보내고 답 올 때까지 다른 일 하기"(비동기).
Dev2 노드들은 DB·LLM·subprocess를 [await]하므로 비동기 라인이라 ainvoke를 써야 한다.

핵심: 노드 함수가 [async def]면 [await graph.ainvoke(state)]로,
일반 [def]면 [graph.invoke(state)]로 실행한다. 둘 다 초기 state(dict)를 받아
모든 노드를 거친 최종 state(dict)를 반환한다. LangSmith 추적은 [config={"callbacks":[tracer]}]로 주입.

```
[python]
import asyncio
from langchain_core.tracers import LangChainTracer

async def main():
    graph = build_dev2_graph()
    initial: Dev2State = {"task_id": "t-1", "pr_id": "42", "pr_diff": diff_text}
    tracer = LangChainTracer(project_name="dev2-myid")   # CLAUDE.md §3
    final_state = await graph.ainvoke(initial, config={"callbacks": [tracer]})
    print(final_state["final_output"])   # Output Contract A (Dev4로 전달)

asyncio.run(main())
```

---

## 3rd-Project-sub2 프로젝트에서

파일: [dev2/nodes/*.py] (각 노드), 그래프 조립부 (예: [dev2/graph.py] 신규)
역할: Dev2는 "Parallel Independent Agent"지만 내부적으로는 StateGraph 선형 파이프라인.
run_tools가 CLI 도구를 병렬 실행 -> vector_search가 pgVector로 과거 유사 이슈 주입
-> llm_judge가 심각도 판정 -> escalate가 Critical 알림 -> store가 PostgreSQL+BigQuery 적재 후
Output Contract A([final_output])를 만들어 Dev4 Aggregator로 넘긴다.

---

## 복습 퀴즈 (3문항)

1. 노드 함수가 [{"static_raw": [...]}]만 반환했는데 [pr_diff] 필드가 사라지지 않는 이유는?
2. async def 노드로 이뤄진 그래프를 [graph.invoke()]로 실행하면 어떤 문제가 생기나?
3. [add_edge(START, "run_tools")]를 빠뜨리면 compile() 시점에 무슨 일이 일어나는가?

### 정답 메모

1. LangGraph가 반환 dict를 기존 state에 얕은 병합(merge)하기 때문. 반환하지 않은 키는 보존된다.
2. 코루틴이 await되지 않아 노드가 실제로 실행되지 않거나 RuntimeWarning/에러 발생. async 그래프는 ainvoke 필수.
3. 진입점(entry point)이 없어 그래프 검증 단계에서 컴파일 에러가 난다.